Bitácora de uso de IA

Proyecto: AutoFix Pro

Equipo 6: Jesus Eduardo Moreno Osoria (2136063) · Osvaldo Salvador Cárdenas Mejorado (2141553) · Abraham Sandoval De León (2136604)

Herramienta: Claude (Anthropic). Las iteraciones 1 a 4 corresponden a la preparación (corrección de los documentos de la Unidad 2) y las 5 a 15 a esta unidad, en el orden de las etapas de la guía. Al inicio de cada prompt se indica entre paréntesis qué integrante lo escribió; los prompts se transcriben resumidos. Cuando el integrante hizo una pregunta de seguimiento sobre la respuesta, también se registra, porque ahí es donde se ve la revisión del equipo. La última columna dice qué aceptamos, qué cambiamos y qué descartamos.

Reparto de etapas en el equipo

Jesús: Etapa 1 (casos de uso) y Etapa 2 (elección del patrón y tabla comparativa).

Abraham: Etapa 3 (mapa de dominio), Etapa 4 (Service Layer) y Etapa 5 (Repository y Unit of Work).

Osvaldo: Preparación de la Unidad 2, Etapa 6 (CQRS), Etapa 7 (SOLID), código base y pruebas, Etapa 8 (diagramas UML y ADR) y Etapa 9 (repositorio, bitácora y matriz).

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	**Preparación U2** Revisar la consistencia de los documentos de la Unidad 2	**(Osvaldo)** «Te paso el BRD, las User Stories, el Glosario y el BPMN de AutoFix Pro. Analízalos con calma y dime qué no cuadra entre ellos; fíjate sobre todo en el BPMN, que es donde creemos que hay más errores.»	Lista de errores del BPMN (carril del Gerente vacío, tareas duplicadas, flujo de refacciones desconectado, bucle de espera, entrega antes del pago y «factura», que está fuera de alcance) y 18 inconsistencias entre documentos.	Supuso que el profesor aceptaría pools de caja negra para los participantes externos. Dejó abierta la duda de si el diagnóstico se cobra.	Revisamos la lista contra los documentos y confirmamos los errores. Decidimos rehacer primero el BPMN, porque los demás documentos dependen del flujo, y después corregir BRD, User Stories y Glosario.
2	**Preparación U2** Generar el BPMN corregido	**(Osvaldo)** «Haz el BPMN corregido como imagen para pasarlo después a Lucidchart. Corrige todo lo que encontraste, que no queden bucles y usa los símbolos de tiempo, documento, notas y mensajes.»	Diagrama 1 (proceso principal en 6 fases) y Diagrama 2 (retiro anticipado), en PNG y PDF.	El primer PNG pesaba más de lo permitido para enviarse y tenía etiquetas encimadas; se corrigió antes de usarlo.	Aceptamos los dos diagramas después de revisar que cada tarea estuviera en el carril de su actor y que la entrega quedara después del pago.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
3	**Preparación U2** Aclarar los temporizadores del BPMN	**(Osvaldo)** «¿Por qué de «Esperar decisión de autorización» salen dos flechas, si eso no puede pasar al mismo tiempo? Y el fin «Cliente notificado del retraso», ¿termina todo el proceso?»	Explicó los eventos de tiempo de borde no interruptores: el aviso sale y la espera continúa. También explicó que la solicitud de compra se queda en el carril del Técnico por la US-010.	Error que detectamos nosotros: los eventos de fin de los avisos parecían cerrar la orden. Se renombraron «Aviso enviado (la espera / la compra sigue)» y se ampliaron las notas.	Mantener los temporizadores no interruptores, porque el aviso no cancela la espera del cliente ni la compra, y dejar la solicitud de compra en el carril del Técnico.
4	**Preparación U2** Corregir el BRD, las User Stories y el Glosario	**(Osvaldo)** «Ahora corrige el BRD, las User Stories y el Glosario para que todo coincida con el BPMN nuevo. Es un proyecto de la universidad, no quiero complicarlo. Dime qué cambias y por qué.» **(Osvaldo, seguimiento)** «Ya lo platicamos en el equipo: la consulta del cliente va con folio y placas, el asesor captura la decisión de la flotilla, no hay inventario y la regla de $15,000 es solo para la cotización inicial. Ajusta los documentos con eso.»	Documentos v2: 24 US, 43 CA, 14 necesidades y la sección «Estados del sistema» del Glosario. Antes de cerrar, la IA nos hizo cuatro preguntas que le tocaba decidir al equipo.	Supuestos de la IA: el diagnóstico no se cobra si hay rechazo total, el crédito es a 15 días naturales, las formas de pago son efectivo, tarjeta o transferencia, y la Nota de Responsabilidad se firma en papel.	Respondimos las cuatro preguntas: el cliente consulta con folio y placas; el asesor captura la decisión de la flotilla; no habrá módulo de inventario; la regla de $15,000 aplica solo a la cotización inicial. Revisamos los supuestos y los aceptamos.
5	**Etapa 1** Preparar el caso y seleccionar los casos de uso	**(Jesús)** «Te paso la guía de la Unidad 3, la transcripción del video y los documentos corregidos de la U2. Me toca la etapa 1: propón de 3 a 5 casos de uso del proceso principal, con al menos uno con excepción o rechazo. Para cada uno dame actor, objetivo, entrada, reglas y resultado, y dime de qué US y de qué parte del BPMN sale.» **(Jesús, seguimiento)** «¿Por qué no metiste recepción y diagnóstico? Es lo primero que pasa en el taller.»	Cinco casos de uso del tramo cotización → autorización → refacciones → entrega (CU-01 a CU-05), con sus US, CA y fase del BPMN. Sobre la pregunta: recepción y diagnóstico casi no tienen reglas que proteger; el tramo propuesto concentra la regla de $15,000, la autorización de flotilla, las refacciones y la entrega con pago. También sugirió PlantUML para los diagramas.	Supuesto: que recepción y diagnóstico podían quedar fuera sin afectar a los demás casos. Lo revisamos y no es del todo así: CU-01 necesita saber si la orden ya tiene diagnóstico.	Aceptamos los cinco casos de uso. Jesús comprobó que todos aparecen en las User Stories y en el BPMN, así que ninguno es corrección de alcance. Recepción y diagnóstico se representan en el dominio solo en lo que CU-01 necesita (sin diagnóstico no se cotiza). Acordamos usar PlantUML.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
6	**Etapa 2** Elegir el patrón de lógica de negocio	**(Jesús)** «Con base en estas reglas del BRD, los criterios de aceptación y los estados del Glosario, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.» **(Jesús, seguimiento)** «Agrega dos criterios que sean nuestros: lo que nos cuesta a nosotros como equipo y qué tan fácil es probarlo. Y en cada fila pon de qué documento de la U2 sale la evidencia; no quiero que diga solo que un patrón «es más escalable».»	Tabla con 6 criterios (4 de la guía y 2 propios) y la evidencia de la U2 en cada fila. Recomendó Domain Model para las operaciones que cambian datos y lecturas sencillas para las consultas.	Supuso que el costo de tener más clases es aceptable para un equipo de tres. Lo discutimos: Domain Model nos cuesta más aprenderlo, pero con Transaction Script las validaciones de estado se repetirían en CU-03, CU-04 y CU-05.	Domain Model para la escritura y lecturas simples (estilo Transaction Script) en las consultas, delimitado en la decisión argumentada. Descartamos Table Module porque la regla del autorizante de flotilla no tiene un lugar natural ahí. Se registró en ADR-001.
7	**Etapa 3** Construir el mapa de dominio	**(Abraham)** «Revisa este Glosario y estas reglas. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato indica la evidencia que usaste y las dudas que tenemos que validar nosotros.» **(Abraham, seguimiento)** «¿La Solicitud de Compra no debería ser su propio Aggregate? Compras la trabaja por separado.»	3 Aggregates (Orden de Servicio, Cotización y Cliente), Value Objects, 2 servicios del dominio, una política, 14 Domain Events y 6 dudas. Sobre la solicitud, explicó el invariante «En Espera de Refacción mientras haya solicitudes pendientes» (CA-019, CA-022), que involucra a la orden y a la solicitud.	Entre las reglas propuestas venía «la cotización debe estar presentada antes de registrar la decisión». El BPMN muestra ese orden, pero ningún criterio de aceptación lo exige: era una regla inventada.	Descartamos esa regla (la guía prohíbe inventar reglas). Abraham dejó la Solicitud de Compra dentro de la Orden por el invariante, separó la Cotización y modeló el Pago como Value Object. Agregamos al Glosario 4 términos nuevos (Línea de Cotización, Decisión del Cliente, Autorizante y Política de Aprobación Gerencial) sin cambiar los existentes.
8	**Etapa 3** Ampliar el mapa de dominio a todo el BRD	**(Abraham)** «El mapa también nos tiene que servir para el proyecto final. Que cubra todo el alcance del BRD y no solo los cinco casos de uso, y que esté más ordenado.» **(Abraham, seguimiento)** «No, el mapa de dominio es uno solo, sin vistas de detalle aparte. Que los textos se lean y que ninguna línea pase encima de un texto.»	Agregó Vehículo, Empleado, Estado de recepción, Diagnóstico, Falla Adicional, Nota de Responsabilidad y Recibo Interno, y propuso marcar con borde punteado lo que se construirá en el proyecto final. Después unificó todo en un solo diagrama con los 5 Aggregates, una nota de 11 reglas y otra de Domain Events.	Error de la IA: primero dividió el mapa en una vista general y tres detalles, aunque pedimos un solo diagrama. Además salía demasiado ancho y con líneas encima de algunos textos.	Un solo mapa de dominio. Aceptamos el borde punteado. Vehículo y Empleado quedan como Aggregates porque existen fuera de una orden, pero no se detallan en el diagrama de clases. Abraham revisó la imagen final a tamaño real antes de aceptarla.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
9	**Etapa 4** Diseñar la Service Layer	**(Abraham)** «A partir de estos cinco casos de uso, propón las firmas de los Application Services. Separa la coordinación de las reglas del dominio y explica cualquier responsabilidad dudosa. Para los métodos usa los mismos nombres de las US y del BPMN.» **(Abraham, seguimiento)** «En revisar_cotizacion el servicio decide si llama a aprobar o a rechazar. ¿Eso no es una regla del negocio metida en la Service Layer?»	Cinco servicios con un método cada uno (generar_cotizacion, revisar_cotizacion, registrar_decision_del_cliente, crear_solicitud_de_compra, registrar_entrega), con su entrada, colaboradores, salida y reglas que delega. Sobre la duda: el servicio solo traduce lo que eligió el Gerente; que el estado sea válido y que el rechazo traiga observaciones lo valida Cotizacion.	Supuesto: que las alertas al Gerente y a Compras salen como Domain Events. Lo revisamos con el BRD: son alertas internas en el panel del usuario, no correo ni SMS; por eso se modelan como AlertaInterna y no como un servicio externo.	Aceptamos un servicio por caso de uso. Abraham contestó con el equipo las cinco preguntas de la revisión obligatoria; la elección entre aprobar y rechazar se queda en el servicio porque es la intención del usuario, no una regla.
10	**Etapa 5** Separar persistencia y transacciones (Repository y Unit of Work)	**(Abraham)** «Propón las interfaces de Repository en términos del dominio, sin SQL, y la Unit of Work: dónde inicia la transacción, qué cambios incluye y qué pasa si algo falla.» **(Abraham, seguimiento)** «¿Por qué pusiste SQLite? El profe nos va a dar la base de datos del Data Center y todavía no sabemos qué gestor es.»	RepositorioDeOrdenes, RepositorioDeCotizaciones y RepositorioDeClientes, más LectorDeOrdenes solo para lectura, y una tabla por caso de uso con lo que se confirma junto y lo que pasa si hay error. Reconoció que el gestor no lo había decidido el equipo.	Error de la IA: propuso SQLite como base de datos sin que nosotros lo decidiéramos. Supuesto: CU-03 cambia dos Aggregates en una transacción, contra la recomendación de un Aggregate por transacción.	Persistencia en memoria en esta fase y «la base de datos del Data Center (gestor por definir)» en el proyecto final, con las mismas interfaces (ADR-004). Aceptamos que CU-03 confirme dos Aggregates en la misma Unidad de Trabajo porque la autorización debe quedar completa o no quedar (ADR-002).
11	**Etapa 6** Separar Commands y Queries (CQRS)	**(Osvaldo)** «Separa Commands y Queries: al menos dos de cada uno, con su Handler, sus dependencias y el DTO de lectura. Dime también qué parte del caso no necesita esta separación.» **(Osvaldo, seguimiento)** «¿De verdad necesitamos una base de datos aparte para las consultas?»	Cinco Commands (uno por caso de uso) y tres Queries: estado de la orden para el cliente (US-021), solicitudes pendientes para Compras (US-011) y cotizaciones por aprobar para el Gerente (US-005), cada una con su vista. La base de lectura separada la planteó solo como alternativa (CQRS completo).	Supuesto: que las consultas del Gerente de US-017 y US-018 pueden esperar al proyecto final porque no están en los cinco casos de uso. Lo aceptamos porque son tan simples que caben en el mismo lector.	CQRS básico: el mismo almacenamiento para leer y escribir, porque el volumen de un taller no justifica dos bases. Las vistas (DTO) nunca se usan para escribir. US-017 y US-018 se agregan después al mismo LectorDeOrdenes (ADR-003).

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
12	**Entregable 06** Escribir el código base y las pruebas	**(Osvaldo)** «Genera el código base en Python que represente la arquitectura: la estructura de carpetas y el código mínimo que pide el entregable 06. Agrega una prueba por cada criterio de aceptación de los cinco casos de uso.» **(Osvaldo, seguimiento)** «Una duda: ¿esto ya es la aplicación? Todavía no nos toca programar el proyecto final.»	Paquetes dominio, aplicacion e infraestructura con las mismas clases del diseño, composicion.py y 20 pruebas automáticas (17 de casos de uso y 3 de la regla de capas). Aclaró que es el entregable 06, no la aplicación final.	Error detectado por una prueba: la consulta del cliente fallaba si las placas se escribían sin guiones («abc 123 d» contra «ABC-123-D»). Se corrigió en el Value Object Placas.	Dejamos el código solo como esqueleto del diseño, sin interfaz y sin base de datos, y lo explicamos así en el informe y en el README. Osvaldo corrió las 20 pruebas y guardó las capturas como evidencia.
13	**Etapa 7** Revisar la propuesta con SOLID	**(Osvaldo)** «Analiza este diseño y el código base y señala posibles violaciones SOLID. No reescribas todo: dame la evidencia, el riesgo y las alternativas para que nosotros decidamos.» **(Osvaldo, seguimiento)** «Lo de la regla de $15,000: si no va en el servicio, ¿dónde la ponemos para que el monto se pueda cambiar después sin tocar Cotizacion?»	7 hallazgos con evidencia, impacto y cambio. Señaló que la regla de $15,000 podía quedar como condicional en ServicioDeCotizacion y propuso una interfaz del dominio (PoliticaDeAprobacionGerencial) que Cotizacion.generar consulta.	Supuesto: que el monto o el alcance de la regla pueden cambiar. El BRD no lo dice; lo tomamos como un cambio posible, no como una regla nueva, y la regla sigue siendo la misma: solo la cotización inicial mayor a $15,000 pasa por el Gerente.	Movimos la regla a PoliticaDeAprobacionGerencial y lo registramos en ADR-005. Confirmamos los cinco servicios separados y la interfaz LectorDeOrdenes. Dejamos los 7 hallazgos en el informe (la guía pide 5) porque los de O y L confirman decisiones que también hay que defender.
14	**Etapa 8** Representar la arquitectura (UML y ADR)	**(Osvaldo)** «Convierte esta arquitectura validada en PlantUML: un diagrama de clases por capas y dos de secuencia, uno del flujo principal y otro con una regla o excepción. No agregues clases, reglas ni relaciones que no aparezcan en la descripción. Después redacta los ADR de las decisiones que ya tomamos.» **(Osvaldo, seguimiento)** «El de clases está revuelto: la infraestructura sale junto a la aplicación. Acomódalo por capas de izquierda a derecha y revisa que los participantes de las secuencias sean clases que existen en el código.»	Diagrama de clases en tres columnas (Aplicación, Dominio, Infraestructura), secuencia de CU-03 (flujo principal) y de CU-01 (regla de $15,000 y excepción sin diagnóstico), y 5 ADR con contexto, alternativas, decisión, consecuencias y trazabilidad.	En el primer render el diagrama de clases salía demasiado ancho y la infraestructura quedaba al lado de la aplicación. Se verificó que todos los participantes de las secuencias son clases reales del código.	Elegimos CU-03 para el flujo principal porque es el que cambia dos Aggregates, y CU-01 para la excepción porque junta la regla de $15,000 y el rechazo sin diagnóstico. Hicimos 5 ADR (la guía pide 3) porque cada uno tiene consecuencias en el diseño. Osvaldo revisó cada imagen: textos legibles y ninguna línea encima de un texto.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
15	**Etapa 9** Validar y entregar (repositorio, matriz y bitácora)	**(Osvaldo)** «Ayúdanos a armar el repositorio con la estructura del ejemplo PetMarket que nos dio el profe, la matriz de trazabilidad extendida hasta componentes y ADR, y el README. Los entregables van en PDF y queremos una versión editable aparte.» **(Osvaldo, seguimiento)** «Junta los cinco ADR en un solo documento. Y quita del repositorio el de «cómo leer los diagramas»; ese era solo para estudiar nosotros.»	Repositorio con docs/, diagramas/, src/ y tests/; matriz en Excel y PDF (hoja del diseño y hoja de necesidades fuera del diseño detallado); README con instrucciones de revisión; los 5 ADR en un solo PDF.	La primera versión de esta bitácora dejaba la decisión del equipo como «pendiente» en varias filas. Cada integrante revisó y completó las de sus etapas.	Recorrimos los cinco casos de uso sobre los diagramas (tabla de la etapa 9) y corrimos las 20 pruebas antes de entregar. El documento «cómo leer los diagramas» queda fuera de los entregables; lo usamos para preparar la defensa.

Lo que aprendimos al usar la IA

1. La IA propone tecnología que nadie decidió (SQLite). Toda propuesta técnica se revisa contra lo que ya sabemos del proyecto: la base de datos será la del Data Center.
2. La IA puede convertir el orden de un diagrama en una regla (cotización «presentada» antes de la decisión). Antes de aceptar una regla buscamos su criterio de aceptación; si no existe, se descarta.
3. Las imágenes se revisan a tamaño real: en dos ocasiones el primer render tenía textos encimados o era demasiado ancho.
4. Preguntar «¿por qué?» funcionó mejor que pedir todo de nuevo: las preguntas de seguimiento de cada integrante son las que cambiaron el diseño.